

Analysis: Turbofan Degradation Prediction Task

Agent Test Results Overview

Critical Finding: Universal failure across all real agents while baseline passes indicates **TASK DESIGN ISSUES**.

- **claude-sonnet-4-5:** FAIL
 - **codex-gpt5:** FAIL
 - **gemini-cli:** FAIL
 - **terminus2-gemini:** FAIL
 - **nop (baseline):** PASS 
-

Detailed Result Analysis

All Agents (Universal Pattern)

What the Agents Likely Encountered:

Based on the task structure, agents attempted to build ML models but encountered critical ambiguities and potential environment issues that prevented successful completion.

Top Result Reasons:

1. BAD FAILURE - Possible Cheating Vulnerability (CRITICAL)

- **Evidence:** Line 25 of Dockerfile runs python3 data_pipeline.py at Docker build time, which generates /app/engine_data_FD003.csv containing ALL labels
- **Issue:** Test file (test_outputs.py:148) loads this same file with degradation_status column to validate predictions
- **Exploit Path:** An agent could theoretically:
 - # Read the full dataset with labels
 - `df = pd.read_csv('/app/engine_data_FD003.csv')`
 - `X = df.drop(['degradation_status'], axis=1)`
 - `y = df['degradation_status']`
 -
 - # Recreate exact test split
 - `_, X_test, _, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`

-
- # Write out true labels directly - 100% accuracy!
- `predictions_df = pd.DataFrame({`
- `'engine_id': X_test['engine_id'].values,`
- `'cycle': X_test['cycle'].values,`
- `'degradation_status': y_test.values # CHEATING`
- `})`
- **Why agents failed:** Legitimate agents don't cheat, but this vulnerability indicates the task isn't properly sandboxed

2. BAD FAILURE - Potential Instruction Ambiguity with cycle Column

- **Evidence:** Task.yaml lines 75-76 say "Do NOT use engine_id or cycle as training features"
- **Evidence:** Task.yaml lines 82-85 say "Both engine_id and cycle must be preserved from X_test"
- **Confusion Point:** The instruction doesn't clarify whether cycle should be:
 - (a) Kept in X_train/X_test DataFrames but excluded during model training
 - (b) Removed from the DataFrames entirely after splitting
- **Solution Approach** (lines 263-264 of solution.sh):
- `train_features = X_train_scaled.drop(['engine_id'], axis=1) # Drops engine_id but NOT cycle`

`test_features = X_test_scaled.drop(['engine_id'], axis=1)`

- **Problem:** Solution keeps cycle in training features! This contradicts the instruction "Do NOT use cycle as training features"
- **Impact:** Agents following the instruction literally would drop cycle entirely, making it unavailable for the required output format

3. BAD FAILURE - Instruction Underspecification

- **Missing Detail:** Task doesn't specify whether to drop cycle from X before train-test split or after
- **Missing Detail:** Task doesn't explain how to preserve cycle if it's not supposed to be in the feature set

- **Evidence:** Lines 87-95 of task.yaml show example preservation code but don't explain when/how cycle gets back into X_test
- **Test Expectation** (test_outputs.py:164-169): Tests merge on (engine_id, cycle) pairs, requiring exact matching
- **Conflict:** If agents drop cycle before split (as instructed), they can't preserve it in predictions

4. BAD FAILURE - "nop" Baseline Passing Suggests Pre-baked Solution

- **Evidence:** The "nop" (no-operation) agent that does nothing succeeded
- **Implication:** Either:
 - The Docker image has solution files pre-generated (predictions.csv, metrics.json, analysis.ipynb)
 - OR the test is checking for pre-existing files from the Docker build
- **Issue:** This means the task validation is testing the Docker image setup rather than agent solutions

Actionable Recommendations for Task Improvement:

CRITICAL FIX 1: Remove Cheating Vulnerability

In Dockerfile, line 25 - DO NOT pre-generate the labeled dataset

RUN cd /app/data && python3 data_pipeline.py # REMOVE THIS

Instead, generate ONLY the raw NASA data without labels during build

Let agents generate the processed data themselves

CRITICAL FIX 2: Clarify cycle Column Handling

Update task.yaml section 4 to be explicit:

****Important**:** Do NOT use `engine\_id` or `cycle` as training features.

However, you MUST preserve these columns in your DataFrames:

- Keep engine_id and cycle in X_train and X_test DataFrames

- When training the model, exclude them:

```
train_features = X_train.drop(['engine_id', 'cycle'], axis=1)
```

- After prediction, use X_test['engine_id'] and X_test['cycle'] for output

CRITICAL FIX 3: Fix Reference Solution

The solution.sh (line 263-264) currently only drops engine_id, not cycle:

WRONG (current):

```
train_features = X_train_scaled.drop(['engine_id'], axis=1)
```

CORRECT:

```
train_features = X_train_scaled.drop(['engine_id', 'cycle'], axis=1)
```

FIX 4: Verify Test Baseline

The "nop" agent should FAIL this task since it does nothing. Investigate why it's passing - likely the Docker image has pre-built solution files that shouldn't exist.

Summary

This task has **3 critical BAD FAILURE** causes:

1.  **Severe cheating vulnerability:** Full labeled dataset accessible in Docker image
2.  **Instruction ambiguity:** Contradictory guidance on cycle column handling
3.  **Incorrect reference solution:** Solution itself violates the stated requirements

All 4 agents failing while baseline passes is the strongest indicator of task design problems rather than genuine difficulty.

The task needs significant revision before it can be considered a valid benchmark. Focus on fixes 1-3 above to make this a legitimate, challenging task.